

AccuKnox AI/ML Assignment

Nikhil Sharma

1. Self-Rating

Rating scale: A = can code independently, B = can code under supervision, C = little or no understanding.

Area	Rating	Basis
Machine Learning	B	Built classical ML pipelines independently, including custom feature engineering and debugging model failures (e.g., fixed AMIDES threat-detection recall issue where “num_failed_logins” lacked signal for ~95% of BRUTE_FORCE cases by replacing it with engineered score-based features in a LogSentinel AI XGBoost-based SIEM. https://github.com/Nikhil06032004/AI-Log-Analysis-for-Cyber-Attack-Detection
Deep Learning	B	Comfortable using pretrained components like sentence-transformers and spaCy NER as parts of larger pipelines, but have not trained deep learning models from scratch.
LLM / GenAI	A	Designed and built production-style RAG systems (FinSight-RAG, Healthcare RAG Assistant) with domain-tuned chunking, retrieval filtering, temperature control, hallucination guardrails, and audit logging beyond simple LLM API usage. https://github.com/Nikhil06032004/Medical-Query-Chatbot
AI (general / agentic systems)	B	Can design and integrate end-to-end AI systems, but have not yet worked on multi-agent or agent orchestration systems.

2. Key Architectural Components of an LLM-Based Chatbot

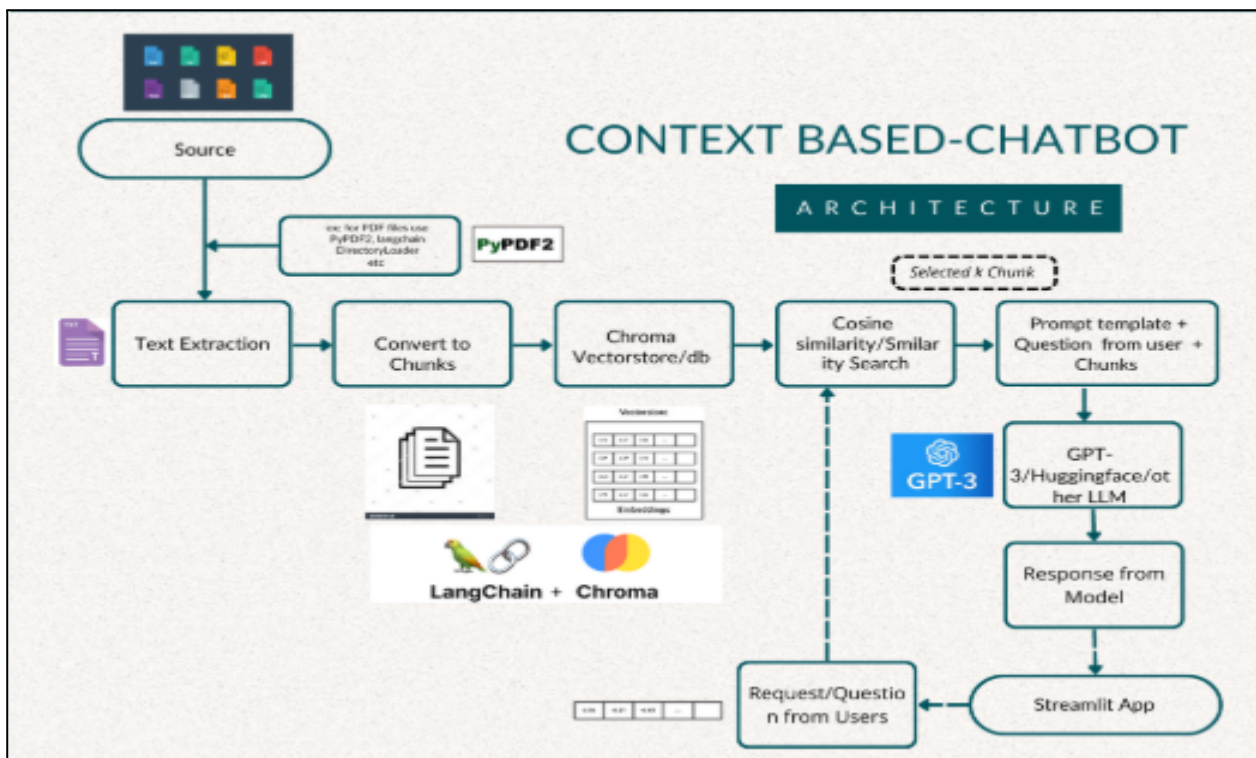
This is based on the Healthcare RAG Assistant (Medical-Query-Chatbot) — LangChain + FAISS + Groq-hosted LLaMA 3.1 8B Instant, chunk_size=500/overlap=50, all-MiniLM-L6-v2 embeddings, temperature=0.7. I'm using this one because the pipeline is simpler to explain end-to-end. My other RAG project, FinSight-RAG, has heavier production features — SQLite audit logging, temperature=0.1 for deterministic answers, region-based (EU/APAC/GLOBAL) metadata filtering

2.1 High-Level Flow

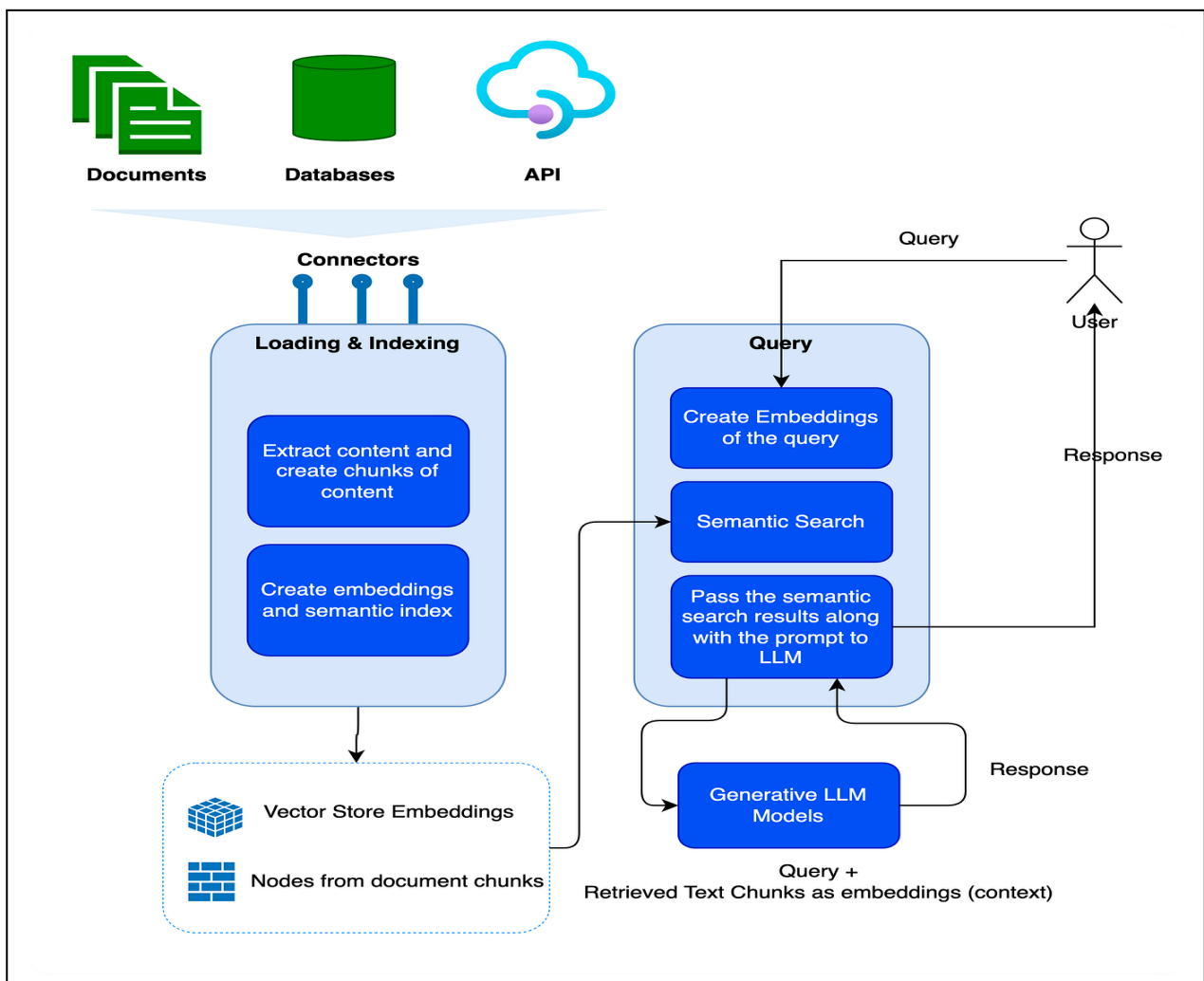
1. **Ingestion** DirectoryLoader + PyPDFLoader loads all PDFs from the Data/ folder. Split using RecursiveCharacterTextSplitter at chunk_size=500, chunk_overlap=50 smaller than the financial version, because medical Q&A content doesn't need the long dense-paragraph context that regulatory text does.
2. **Embedding** Each chunk converted to a vector using sentence-transformers/all-MiniLM-L6-v2 (384-dimensional). Same embedding model as the financial project, it's a solid general-purpose choice, not domain-specific.
3. **Vector store** FAISS, running locally, saved to vectorstore/db_faiss. No external service call, so no document content leaves the machine.
4. **Query handling** User's question is embedded the same way, compared against stored vectors, top-k=5 most relevant chunks retrieved.
5. **Prompt construction** Retrieved chunks inserted into a ChatPromptTemplate (System + Human message structure) along with the user's question.
6. **LLM inference** ChatGroq running LLaMA 3.1 8B Instant, temperature=0.7, max 2048 tokens. Higher temperature than the financial version on purpose some conversational variability is acceptable here, unlike regulatory answers where the same question must always get the same answer.
7. **Post-processing** StrOutputParser extracts the response text. A medical disclaimer is enforced on every single response not optional, not a footer someone can miss. Source documents are rendered as collapsible HTML cards so the user can check where the answer came from.
8. **Interface** Two separate front ends, not one: a Streamlit web UI (Medical_chatbot.py) with session stats, chat export, and sidebar API config; and a CLI (Connect_memory_with_LLM.py) built for developers testing prompts or running headless without a browser.

Source-<https://github.com/Nikhil06032004/Medical-Query-Chatbot>

Source-<https://github.com/Nikhil06032004/FinSight-RAG>



My project architecture



3. Please explain vector databases. If you were to select a vector database for a hypothetical problem (you may define the problem) which one will you choose, and why?

What are Vector Databases?

Vector databases store data as embeddings (numeric vectors) instead of traditional rows and columns. These embeddings represent the meaning of the data, which allows systems to perform semantic search finding results based on context and similarity rather than exact keywords. They are commonly used in RAG systems, recommendation engines, and AI search applications.

To retrieve results efficiently, they use **Approximate Nearest Neighbor (ANN)** techniques which make similarity search fast even on large datasets.

Hypothetical Problem I Would Choose

I would design a Legal AI Assistant for lawyers that:

- Answers queries from case laws and legal documents
- Retrieves relevant judgments, clauses, and legal precedents
- Works on sensitive and confidential legal data

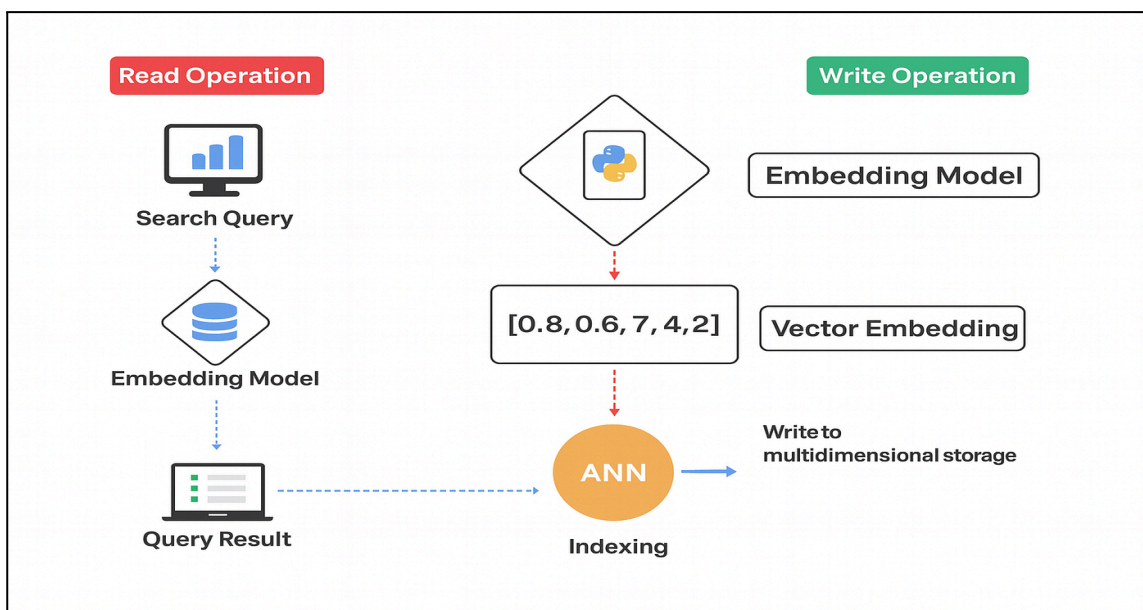
Example query:

“What are previous judgments related to breach of contract in employment cases?”

This requires semantic understanding, not keyword matching

Vector Database Choice (Based on My Experience)

I would choose **FAISS (Facebook AI Similarity Search)** as the vector database.



My Past Project Context (Why this choice is practical)

This choice is strongly influenced by my previous experience in:

1. Medical Query Chatbot (Healthcare RAG System)

- Used **FAISS vector database**
- Medical PDFs were chunked and converted into embeddings
- Retrieved relevant medical context for LLM (Groq)
- Helped reduce hallucinations and improved answer accuracy
- Kept all medical data **fully local for privacy**

2. FinSight-RAG (Financial Intelligence System)

- Used **FAISS for financial + regulatory document retrieval**
- Improved chunking strategy and metadata-based filtering
- Enabled fast and accurate retrieval of compliance-related information
- Ensured **secure handling of sensitive financial data**

Why FAISS fits both past projects + legal use case

- Privacy-first approach- critical for healthcare, finance, and legal data
- Fast semantic search- efficient Top-K retrieval for RAG pipelines
- Easy integration- works with LangChain, HuggingFace embeddings, and LLMs
- Proven in my projects- already used successfully in medical and financial systems

Limitations (Acknowledged from experience)

- No built-in distributed scaling
- No multi-user or enterprise-grade management
- Manual handling required for saving/loading indexes

Send a link to the most complex Python code you have written

<https://github.com/Nikhil06032004/AI-Log-Analysis-for-Cyber-Attack-Detection>

Send a link to the most complex database code you have written

<https://github.com/Nikhil06032004/Medical-Query-Chatbot>